

Министерство науки и высшего образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ
по лабораторной работе №4
по курсу «Логика и основы алгоритмизации в инженерных задачах»
на тему «Бинарное дерево поиска»

Выполнили:

студенты группы 24ВВВ3

Алмакаев В.В.

Буров А.О.

Приняли:

к.т.н., доцент Юрова О.В.

к.т.н., Деев М.В.

Пенза 2025

Цель работы – Изучить структуру данных «бинарное дерево поиска», реализовать базовые операции (добавление элементов, визуализация дерева, поиск количества вхождений заданного значения) и приобрести практические навыки работы с рекурсивными алгоритмами при обработке древовидных структур.

Лабораторное задание:

Задание

1. Реализовать алгоритм поиска вводимого с клавиатуры значения в уже созданном дереве.
2. Реализовать функцию подсчёта числа вхождений заданного элемента в дерево.
3. * Изменить функцию добавления элементов для исключения добавления одинаковых символов.
4. * Оценить сложность процедуры поиска по значению в бинарном дереве.

4.*

Ввод данных: $O(n)$

Построение дерева: $O(n \log n)$ в среднем, $O(n^2)$ в худшем

Вывод дерева: $O(n)$

Поиск количества: $O(n)$

Поиск уровня: $O(\log n)$ в среднем

Код программы

C#

```
using System;

class Program
{
    class Node
    {
        public int Data;
        public Node Left;
        public Node Right;

        public Node(int data)
        {
            Data = data;
            Left = null;
            Right = null;
        }
    }

    static Node root = null;

    static Node CreateTree(Node node, int data, int level, ref bool elementAdded)
```

```

{
    if (node == null)
    {
        elementAdded = true;
        return new Node(data);
    }

    if (data < node.Data)
    {
        node.Left = CreateTree(node.Left, data, level + 1, ref elementAdded);
    }
    else if (data > node.Data)
    {
        node.Right = CreateTree(node.Right, data, level + 1, ref elementAdded);
    }
    else
    {
        if (level >= 3)
        {
            Console.WriteLine($"Элемент {data} уже существует. Введите другое
значение:");
            int newData = int.Parse(Console.ReadLine());
            root = CreateTree(root, newData, 0, ref elementAdded);
            return node;
        }
        else
        {
            node.Right = CreateTree(node.Right, data, level + 1, ref element-
tAdded);
        }
    }

    return node;
}

static Node SearchTree(Node node, int value)
{
    if (node == null) return null;
    if (node.Data == value) return node;
    return value < node.Data ? SearchTree(node.Left, value) : Search-
Tree(node.Right, value);
}

static int CountOccurrences(Node node, int value)
{
    if (node == null) return 0;
    int count = node.Data == value ? 1 : 0;
    return count + CountOccurrences(node.Left, value) + CountOccur-
rences(node.Right, value);
}

static void PrintTree(Node node, int level)
{
    if (node == null) return;
    PrintTree(node.Right, level + 1);
    Console.WriteLine(new string(' ', level * 3) + node.Data);
    PrintTree(node.Left, level + 1);
}

static void InputTree()
{
    root = null;

    Console.Write("Введите количество элементов в дереве: ");

```

```

int n = int.Parse(Console.ReadLine());

int count = 0;
Console.WriteLine($"Введите {n} элементов:");
while (count < n)
{
    int value = int.Parse(Console.ReadLine());
    bool elementAdded = false;
    root = CreateTree(root, value, 0, ref elementAdded);

    if (elementAdded)
    {
        count++;
    }
}

static void ShowMenu()
{
    Console.WriteLine("\n=== БИНАРНОЕ ДЕРЕВО ПОИСКА ===");
    Console.WriteLine("1. Создать дерево");
    Console.WriteLine("2. Показать дерево");
    Console.WriteLine("3. Поиск элемента");
    Console.WriteLine("4. Подсчёт вхождений");
    Console.WriteLine("0. Выход");
    Console.Write("Выберите действие: ");
}

static void Main()
{
    Console.OutputEncoding = System.Text.Encoding.UTF8;

    while (true)
    {
        ShowMenu();
        string choice = Console.ReadLine();

        switch (choice)
        {
            case "1":
                InputTree();
                Console.WriteLine("Дерево создано!");
                break;

            case "2":
                if (root == null) Console.WriteLine("Дерево пустое!");
                else
                {
                    Console.WriteLine("\nДерево:");
                    PrintTree(root, 0);
                }
                break;

            case "3":
                if (root == null)
                {
                    Console.WriteLine("Дерево пустое!");
                    break;
                }
                Console.Write("Введите значение для поиска: ");
                int searchValue =

int.Parse(Console.ReadLine());

```

```

        Console.WriteLine(SearchTree(root, searchValue) != null ?
$"Значение {searchValue} найдено!" : $"Значение {searchValue} не найдено.");
        break;

    case "4":
        if (root == null)
        {
            Console.WriteLine("Дерево пустое!");
            break;
        }
        Console.Write("Введите значение для подсчёта: ");
        int countValue = int.Parse(Console.ReadLine());
        int occurrences = CountOccurrences(root, countValue);
        Console.WriteLine($"Элемент {countValue} встречается {occurrences} раз(a)");
        break;

    case "0":
        Console.WriteLine("Выход...");
        return;

    default:
        Console.WriteLine("Неверный выбор!");
        break;
    }
}
}
}

```

Результаты работы программы

```

3. Поиск элемента
4. Подсчёт вхождений
0. Выход
Выберите действие: 1
Введите количество элементов в дереве: 5
Введите 5 элементов:
1
2
3
4
5
Дерево создано!

=== БИНАРНОЕ ДЕРЕВО ПОИСКА ===
1. Создать дерево
2. Показать дерево
3. Поиск элемента
4. Подсчёт вхождений
0. Выход
Выберите действие: 2

Дерево:
      5
     /
    4
   /
  3
 /
2
/
1

```

Рисунок 1 - Результат работы программы

Вывод: В ходе работы была успешно реализована структура бинарного дерева поиска. Освоены основные операции: добавление элементов с сохранением свойств дерева, визуализация структуры и рекурсивный поиск количества вхождений значения.